CCA

Next.js 16 Foundations

Building Modern Full-Stack Applications with React

Program: CCA Full-Stack Developer Pathway

Session: 01 - App Router, Layouts, Server vs Client Components

Instructor: Sayuru Amarasinghe

Instructor's Note: If React helps us build UI, Next.js helps us build real-world applications faster, better, and in a more structured way.

NotebookLM

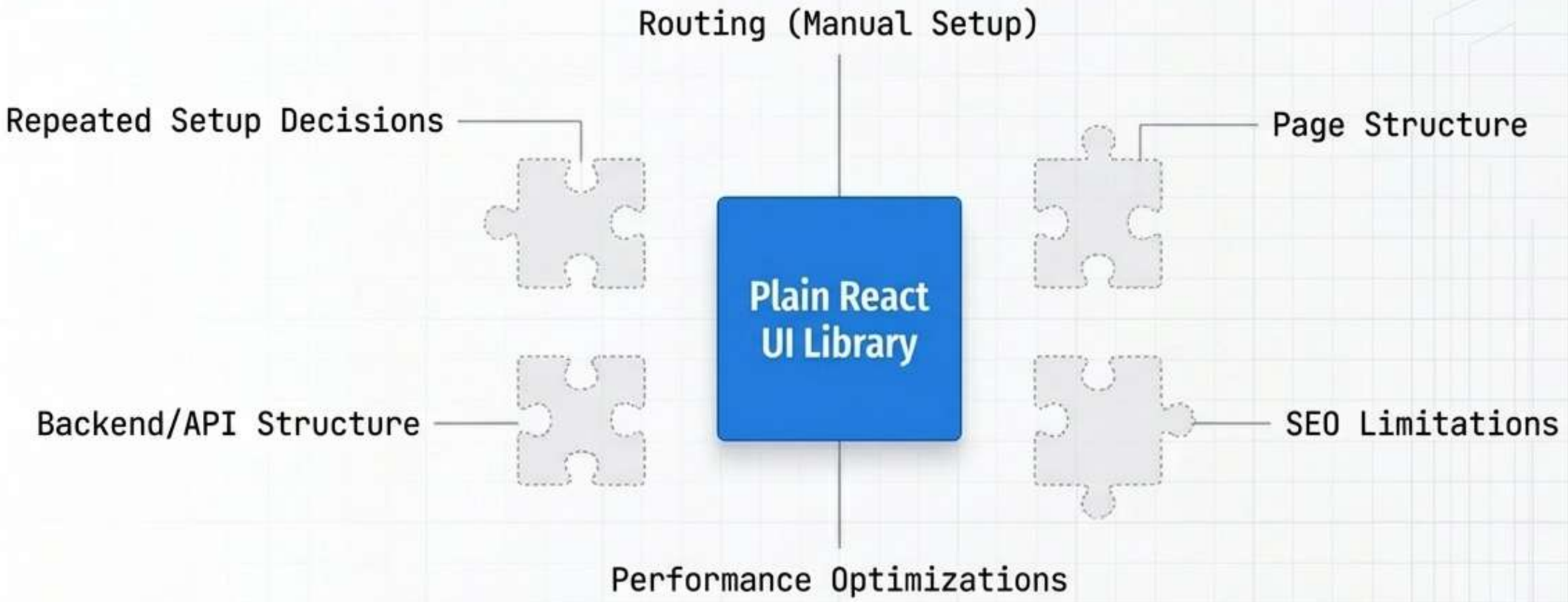
Session Roadmap

1. What Next.js is
2. Why Next.js > Plain React
3. The App Router mechanics
4. Pages & Layouts creation
5. Routing fundamentals
6. Dynamic routes
7. Server vs. Client Components
8. Live Project Structure Build

Mental Model: By the end of today, you should not just know how to run a Next.js project. You should understand the basic mental model behind how modern Next.js apps are built.

NotebookLM

The Problem with Plain React Apps

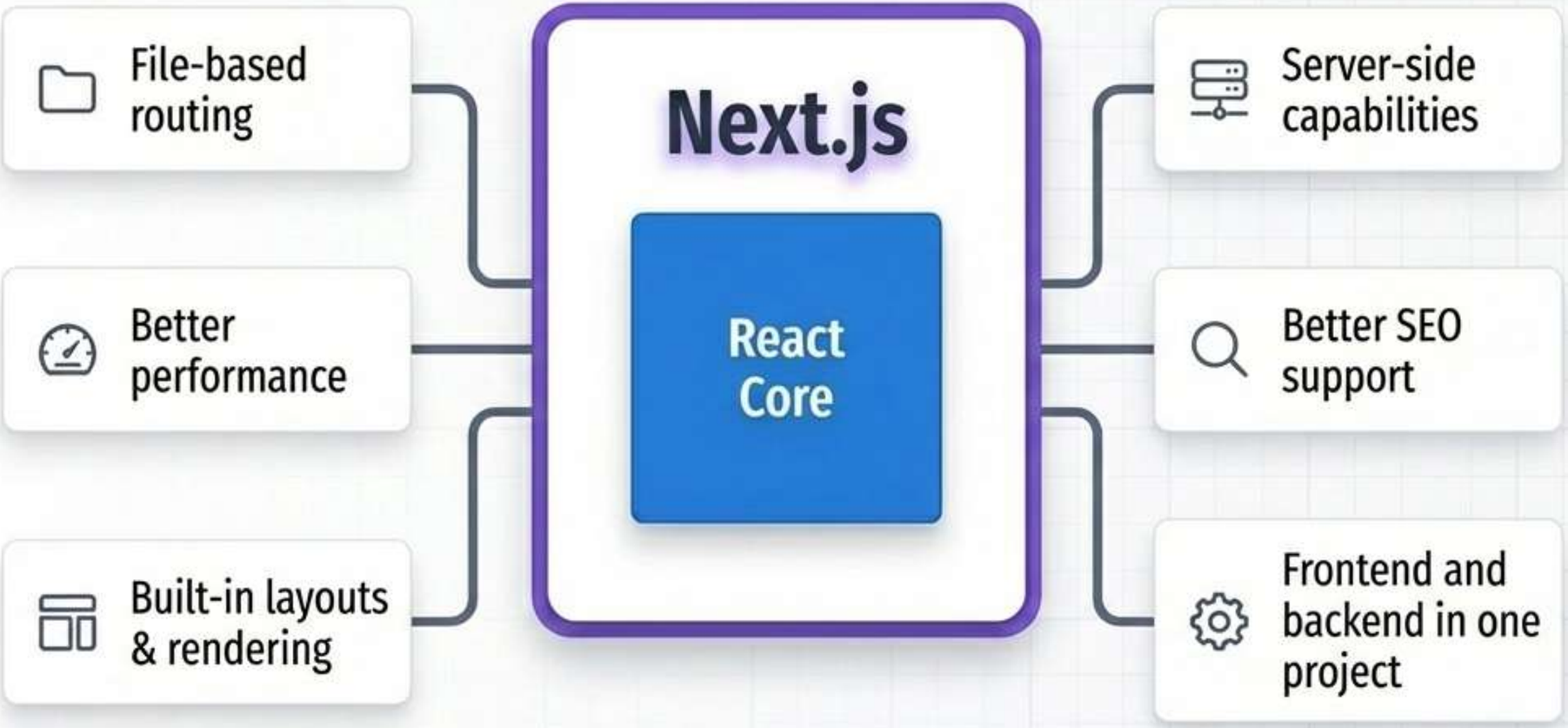


Synthesis: React is powerful, but building complete production apps often needs more structure. Next.js does not replace React; it builds on top of it.

NotebookLM

The All-In-One Framework Wrapper

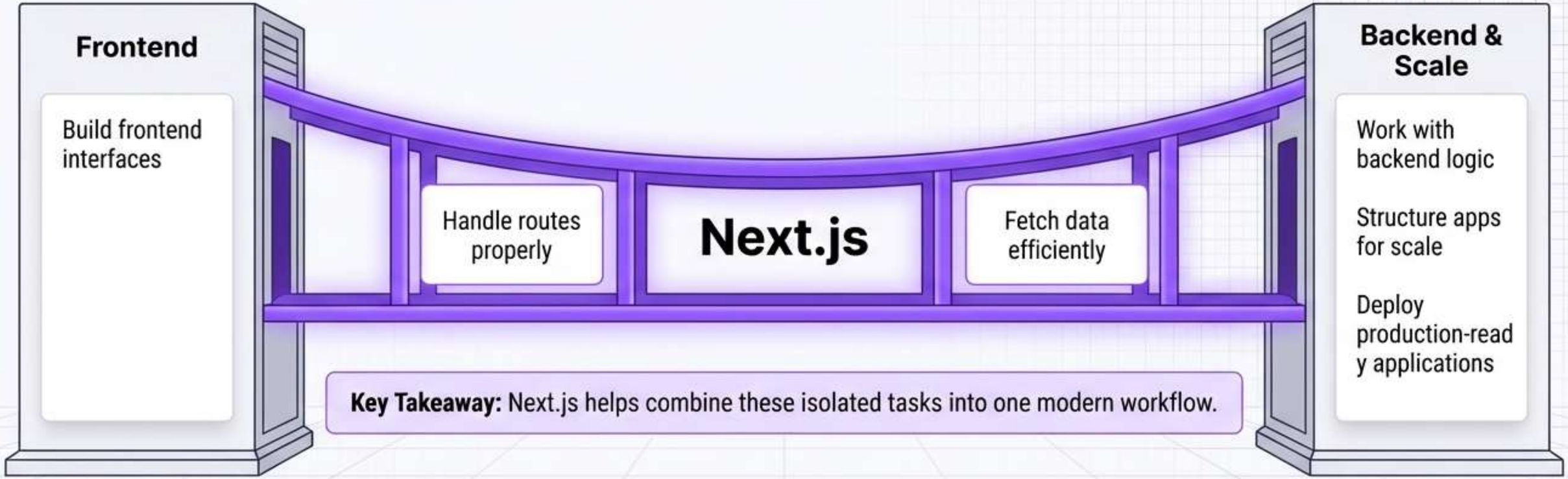
Next.js is a React framework for building full-stack web applications.



> **Instructor's Insight:** This is why startups, SaaS products, dashboards, and scalable platforms use Next.js. It provides the structure and speed we otherwise have to build manually.

NotebookLM

Why Next.js Matters for Full-Stack Developers



Key Takeaway: Next.js helps combine these isolated tasks into one modern workflow.

→ Mental Model Shift: As a full-stack developer, you should not only think in components. You should think in pages, routes, data, rendering, and scalability.

NotebookLM

The Paradigm Shift: React vs Next.js

React	Next.js
Type: Library for building UI	**Type:** Framework built on React
Setup: Flexible but needs manual configuration	**Setup:** Better defaults for performance & structure
Routing: Added separately	**Routing:** Built-in routing out of the box
Backend: Handled in a separate project	**Backend:** Full-stack features included

The Paradigm Shift: React teaches component-based UI thinking. Next.js teaches **application thinking.**

NotebookLM

The Modern Next.js Mental Model

SERVER

1. Routes
2. Pages & Layouts
3. Server-rendered logic & Components

CLIENT

4. Client-side interactivity (Only where needed)

Instructor's Warning: Do not treat this exactly like old React projects. In the Next.js App Router, server-first thinking is the default mindset.

The Golden Rule: Build on the server by default. Use the client only when interaction is needed.

NotebookLM

The App Router Mechanism

The App Router uses folders and files to define routes.

```
app/
├── page.tsx
├── about/
│   └── page.tsx
└── jobs/
    ├── page.tsx
    └── [id]/
        └── page.tsx
```

→ yoursite.com/
Home page

→ yoursite.com/about
About page

→ yoursite.com/jobs
Jobs page

→ yoursite.com/jobs/123
Dynamic job details

Instructor's Note: The folder structure itself becomes part of how your app works. This is why Next.js feels structured and clean.

NotebookLM

Core File Anatomy

app/

- page.tsx — Represents a route/page.
- layout.tsx — Shared layout across multiple pages.
- loading.tsx — Loading UI for a route segment.
- not-found.tsx — Custom not-found page.
- route.ts — Backend route handler / API endpoint.

Today's Focus:

We will mainly focus on page.tsx and layout.tsx. Later sessions will expand into loading, APIs, and advanced features.

NotebookLM

The Great Divide: Server vs Client Components

Server Components

- Default in App Router
- Good for data fetching
- Better for performance
- Keep heavy logic on the server

Client Components

- Used for user interactivity
- Needed for useState, useEffect, click handlers, browser APIs
- Must explicitly use the `"use client"` directive at the top of the file

Bottom Line: Not everything should be a client component. Just because we are used to React doesn't mean the app should become entirely client-side.

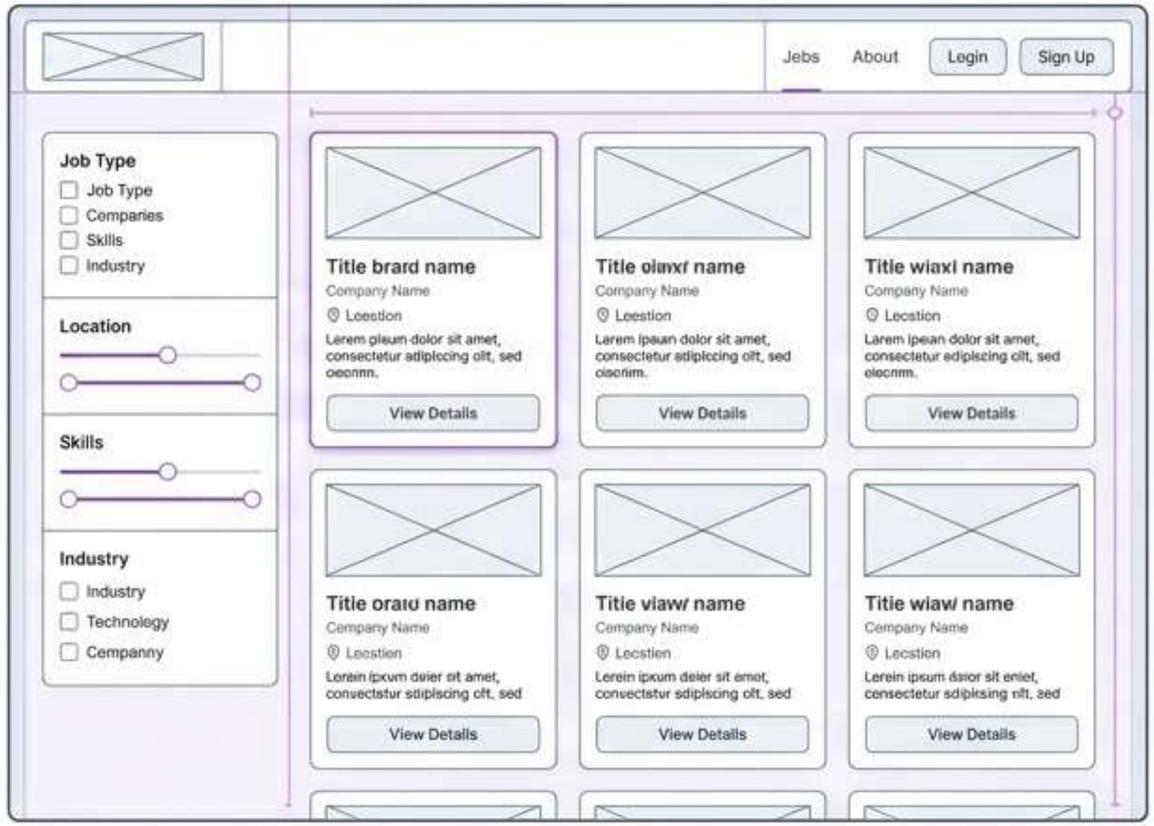
NotebookLM

The Application Blueprint: CCA DevJobs

A scalable job and internship platform built specifically for CCA students.
We learn by building a project that grows with every session.

Phase 1: Today's Build

- [x] Home page & About page
- [x] Jobs page
- [x] Dynamic single job page
- [x] Shared layout and navigation
- [x] Basic filtering/search interaction



Phase 2: Future Sessions

- [ ] Apply form & API routes
- [ ] Database connection
- [ ] Authentication
- [ ] Admin panel & Deployment

NotebookLM

Today’s Output Checkpoints

```
[✓] SUCCESS: A running Next.js 16 project is initialized.
[✓] SUCCESS: Multiple routes are active using the App Router.
[✓] SUCCESS: A shared layout wraps the application.
[✓] SUCCESS: Navigation flows seamlessly between pages.
[✓] SUCCESS: At least one dynamic route is functioning.
[✓] SUCCESS: One interactive client component is successfully isolated.
```

You now have a working understanding of how modern full-stack apps start. Let’s start building.

NotebookLM